

Faculty of Information Technology (FIT) - Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 - Software Testing (AI-augmented, 2026)

1. Student information

- **Student name (printed):** Nguyen An Nghiep
- **Student ID:** 23127234
- **Class / Cohort:** Software Testing - 23KTPM1
- **Assignment ID:** HW06
- **Assignment date:** 19/08/2026
- **AI tool used:** Claude
- **AI used:** Yes

2. Audit table

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
Interaction 1 — Claude, step-by-step generation for Pool A / FR-05. Exact prompts are recorded in Section 2.1.	35 Pool A candidates: 22 VALID, 5 INVALID, 8 INCOMPLETE.	INCOMPLETE	The response achieved the target and covered search partitions, hostile input, and schema checks, but 13 cases used duplicate or unsupported oracles, including assumed trimming, case-insensitivity, wildcard escaping, a two-second SLA, and an incorrect unsupported-method case.	Audited every candidate and corrected all 13 cases in Pool A final cases . Exploratory behavior was separated from normative pass/fail rules.
Interaction 2 — Claude, step-by-step generation for Pool B / FR-10. Exact prompts are recorded in Section 2.2.	35 Pool B candidates: 28 VALID, 3 INVALID, 4 INCOMPLETE.	INCOMPLETE	The state-machine coverage was useful, but seven cases needed correction. The response contradicted the final canceled state, assumed same-state idempotency, used uncertain	Corrected the seven cases in Pool B final cases and added TC-B-36–39 for an expired JWT, missing backward/final-state edges, and

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
			403/404 IDOR behavior, and provided incomplete concurrency/schema steps. It also missed four important cases.	an admin-versus-owner race.
Interaction 3 — Claude, step-by-step generation for Pool C / FR-15. Exact prompts are recorded in Section 2.3.	35 Pool C candidates : 28 VALID, 4 INVALID, 3 INCOMPLETE.	INCOMPLETE	The response covered every main body field and several security partitions, but seven cases relied on requirements that did not exist, such as integer-only prices, URL validation, and unique names. Some property/media-type oracles were also incomplete. It missed the combined role-escalation/prototype-pollution case.	Corrected the seven cases in Pool C final cases and added TC-C-36 for a normal-user token combined with role and __proto__ injection.
Interaction 4 — Claude architecture review. Full prompt: Act as an architecture reviewer for an AI-driven API test generator. Verify whether the following student-designed flow is good or bad. Do not draw the diagram. Explain the role of each	Full AI answer: Verdict: GOOD / VALID foundation. The flow separates specification understanding, coverage generation, expected-result construction, execution planning, duplicate removal, quality control, and output rendering. The Oracle Builder is essential because it turns generated inputs into measurable tests by attaching expected status, response, schema, persisted	VALID	The main responsibilities are separated correctly, the audit gate prevents direct trust in generated cases, and the human loop handles ambiguity. The design is implementable as a reusable skill.	Added a direct requirement-evidence path from the normalized model to the Oracle Builder and clarified each component in Main_report.md and SKILL.md . The student must draw the final diagram manually.

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
component, identify missing information or feedback paths, and give a corrected flow only if necessary. 1. Draw an input box named API specification + selected API. 2. Connect it to Specification loader and then Normalized API model. 3. From the normalized model, draw four parallel boxes: Domain partitions and boundaries, State transitions, Security tests, and Schema validation. 4. Connect all four boxes to Oracle builder. 5. Continue to Fixture planner, then Deduplication, and then AI audit gate. 6. Draw two outputs from the audit gate: Markdown test	state, and forbidden side effects. Without it, the design would be an API-request generator rather than an API-test generator. The following improvements strengthen the design without changing its main structure: 1. Add a direct requirement-evidence arrow from Normalized API model to Oracle builder. The four coverage branches create test ideas, while the normalized model supplies the evidence for their expected results. 2. Allow Human review to send specification-extraction problems back to the loader/model and missing-coverage problems back to the four generators, not only oracle problems to the Oracle Builder. 3. Make the audit gate verify unique IDs, requirement traceability, safe security scope, secret handling, executable setup/cleanup, and output syntax.			

(1) Prompt + Tool	(2) AI Output	(3) Verdict	(4) Reasoning	(5) Student Fix
cases and Postman Collection v2.1. 7. Add a Human review box beside the audit gate. Draw a feedback arrow from human review back to the oracle builder for ambiguous or incorrect candidates. 8. Add small labels beside the audit gate: VALID, INVALID, and INCOMPLETE.	4. Treat only VALID cases as final outputs. Route INVALID and INCOMPLETE cases to correction/human review and then audit them again.			
Check whether the design preserves requirement traceability, creates measurable oracles, supports independent fixtures, prevents duplicated cases, and keeps a human responsible for ambiguous AI output.	The accepted design is documented in Main_report.md and implemented as reusable instructions in api-test-generator/SKILL.md . The final assignment diagram must still be drawn manually by the student.			

2.1 Interaction 1 — Pool A / FR-05 prompt and answer

Prompt — Step 1: extract the endpoint model

You are assisting a Software Testing student as an API test designer. Read the provided EShop API specification and requirements, but do not generate test cases yet. Focus only on FR-05, GET /api/products and its optional search query.

Extract a compact endpoint model containing: method and URL, authentication, every path/query/header parameter, valid and invalid input rules, documented success and error statuses, product response fields and types, relevant SEC-01-SEC-07 rules, and any missing or ambiguous behavior. Do not invent rules. Clearly separate normative requirements from assumptions requiring human review.

Answer — Step 1

Claude identified GET /api/products as a public product-list/search endpoint with one optional search query parameter and the required course header X-Student-Id. It extracted product-list schema expectations and identified SQL injection, information disclosure, unsafe query parsing, and exact schema validation as relevant risks. It also marked case sensitivity, whitespace trimming, Unicode normalization, duplicate parameters, wildcard behavior, unknown parameters, strict additional-property handling, and performance thresholds as unspecified or ambiguous.

Prompt — Step 2: build the coverage model

Using only the FR-05 endpoint model from Step 1, create a coverage checklist before writing test cases. Include domain partitions and boundaries for omitted, empty, whitespace, exact, prefix, middle, suffix, case variants, Unicode, encoding, numeric text, punctuation, very long input, duplicate parameters, and unknown parameters. Include safe SEC-05 SQL-injection/wildcard/error-disclosure probes and exact successful-response schema validation.

For every proposed check, state whether its oracle is normative or requires human confirmation. Avoid destructive payloads and assume a disposable local database at http://localhost:3000.

Answer — Step 2

Claude produced a coverage model spanning positive, negative, boundary, encoding, robustness, security, HTTP-method, and schema categories. It proposed baseline and post-probe smoke checks to detect unintended mutation. The checklist was broad enough for 35 candidates, but it revealed several assumptions that later required correction, especially literal wildcard behavior, case-insensitive equivalence, trimming, strict ordering, and a performance threshold.

Prompt — Step 3: generate the candidates

Generate exactly 35 AI candidate test cases for Pool A / FR-05 from the reviewed coverage checklist. Use IDs TC-A-01 through TC-A-35 and base URL http://localhost:3000. Every request must use X-Student-Id: 23127234.

Use this table only:

ID	Technique / Tag	Objective	Preconditions	Request (Method + URL)	Input	Steps	Expected Status	Expected Result
----	-----------------	-----------	---------------	------------------------	-------	-------	-----------------	-----------------

Make Technique / Tag clearly identify Domain partition, Boundary, Security, or Schema validation. Include observable status, JSON/schema, information-leakage, mutation, and read-back expectations. Preserve ambiguity instead of silently inventing an oracle. Do not add AI Case ID, Requirement, Revision, Oracle Type, or Original Attribution columns.

Answer — Step 3

The complete 35-case answer is stored in [pool_A_FR05_testcases.md](#). It included 22 candidates accepted as VALID, five later classified INVALID, and eight later classified INCOMPLETE. The corrected 35-case result is stored in [pool_A_FR05_final_testcases.md](#).

2.2 Interaction 2 — Pool B / FR-10 prompt and answer

Prompt — Step 1: extract the state machine

Read the supplied EShop API specification and requirements. Do not generate test cases yet. Focus on FR-10 order cancellation and admin order-status management.

Extract the complete state model for pending, confirmed, shipping, delivered, and canceled. List every documented valid transition, forbidden transition, final state, cancellation rule, actor and role, ownership rule, authentication rule, path/body parameter, status code, response schema, and supporting API needed for setup or read-back. Mark unclear 401/403/404, idempotency, and concurrency behavior as ambiguous instead of guessing.

Answer — Step 1

Claude extracted the normal path **pending** → **confirmed** → **shipping** → **delivered**, cancellation from early states, terminal-state rules, owner cancellation, administrator status changes, bearer authentication, ownership/IDOR concerns, and read-back requirements. It correctly recognized that independent order fixtures were necessary. Some policies—especially non-disclosing 403 versus 404 behavior, same-state updates, and concurrent response combinations—remained ambiguous.

Prompt — Step 2: build state, security, and schema coverage

Build a coverage matrix for FR-10 before generating cases. Cover the complete valid and invalid state-transition matrix, cancellation rules, final states, backward and skipped edges, repeated operations, path-ID partitions, missing, empty, Basic, tampered, and malformed authentication, IDOR, role enforcement, SQL/information-disclosure safety, response schema, persisted-state read-back, isolation from unrelated orders, and one synchronized concurrency scenario.

Indicate which expected results come directly from the specification and which need human confirmation. Do not chain unrelated cases onto the same order.

Answer — Step 2

Claude created a broad matrix covering owner cancellation, administrative updates, invalid paths, authentication, authorization, state persistence, final states, and concurrency. The model still omitted expired-token coverage, **confirmed** → **pending**, **delivered** → **canceled**, and the combined admin-confirmation/owner-cancellation race later added by the student.

Prompt — Step 3: generate the candidates

Generate exactly 35 AI candidate test cases for Pool B / FR-10 using IDs TC-B-01 through TC-B-35 and base URL <http://localhost:3000>. Every request must carry X-Student-Id: 23127234 and the actor-specific bearer token when required.

Use this table only:

ID	Technique / Tag	Objective	Preconditions	Request (Method + URL)	Input	Steps	Expected Status	Expected Result
----	-----------------	-----------	---------------	------------------------	-------	-------	-----------------	-----------------

Tag cases clearly as Domain partition, State transition, Security, or Schema validation. Each case must use an independent fixture and verify the persisted state and unrelated-order isolation. Keep uncertain oracles visible for human review. Do not add audit-only columns.

Answer — Step 3

The complete 35-case answer is stored in [pool_B_FR10_testcases.md](#). Human review accepted 28 candidates and corrected seven. Four student-designed extensions produced a final set of 39 cases in [pool_B_FR10_final_testcases.md](#).

2.3 Interaction 3 — Pool C / FR-15 prompt and answer

Prompt — Step 1: extract the creation contract

Read the supplied EShop API specification and requirements. Do not generate test cases yet. Focus on FR-15, POST /api/products.

Extract the endpoint method/URL, required administrator authentication, request fields and types, required versus optional fields, name and price constraints, category rules, success/error statuses and schemas, server-managed properties, and relevant SEC-01-SEC-07 risks. Identify any missing rules for decimals, imageUrl format, duplicate names, extra properties, and media types without inventing them.

Answer — Step 1

Claude identified required `name`, positive numeric `price`, and existing `category_id`, together with optional `description` and `imageUrl`, administrator-only creation, response/read-back schema, cleanup, property injection, role escalation, SQL injection, and XSS-safe data handling. It also identified ambiguity around decimal prices, empty optional strings, URL validation, uniqueness, unsupported media type, and extra-property policy.

Prompt — Step 2: build parameter and security coverage

Create a coverage checklist for every POST `/api/products` parameter before generating cases. Include missing, null, empty, whitespace, minimum, maximum, above maximum, wrong type, Unicode, positive/zero/negative/decimal price, existing/nonexistent/zero/negative/wrong-type category, optional-field variants, unknown and protected properties, malformed JSON/media type, missing and normal-user authorization, SQL/XSS data, and exact create/read-back schema.

Require unique fixtures, unrelated-product integrity, cleanup, and safe local security payloads. Mark unsupported business policies as human-review items.

Answer — Step 2

Claude generated a coverage model for all major body fields, authentication actors, malformed content, extra properties, injection, schema, persistence, and cleanup. The model was extensive but treated several optional policies as requirements, which later caused invalid or incomplete candidates.

Prompt — Step 3: generate the candidates

Generate exactly 35 AI candidate test cases for Pool C / FR-15 using IDs TC-C-01 through TC-C-35 and base URL `http://localhost:3000`. Every request must carry X-Student-Id: 23127234, Content-Type, and the appropriate admin, user, or missing Authorization profile.

Use this table only:

ID	Technique / Tag	Objective	Preconditions	Request (Method + URL)	Input	Steps	Expected Status	Expected Result
----	-----------------	-----------	---------------	------------------------	-------	-------	-----------------	-----------------

Tag cases clearly as Domain partition, Boundary, Security, or Schema validation. For accepted creates, capture the ID, read the product back, verify types and values, protect unrelated products, delete the fixture, and verify cleanup. For rejected creates, verify that no row was inserted. Keep ambiguous policies open for human review and do not add audit-only columns.

Answer — Step 3

The complete 35-case answer is stored in [pool_C_FR15_testcases.md](#). Human review accepted 28 candidates and corrected seven. The student added TC-C-36, giving 36 final cases in [pool_C_FR15_final_testcases.md](#).

3. Summary of AI accuracy

3.1 Interaction-level verdicts

Metric	Count	Percentage
Total AI interactions audited	4	100%
VALID (correct and accepted)	1	25%
INVALID (wrong and rejected)	0	0%
INCOMPLETE (useful but required human work)	3	75%

3.2 Generated-testcase verdicts

Metric	Count	Percentage
Total AI-generated candidates	105	100%
VALID	78	74.29%
INVALID	12	11.43%
INCOMPLETE	15	14.29%
INVALID + INCOMPLETE corrected by the student	27	25.71%

4. Conclusion

Claude was effective at producing broad API coverage quickly, but the three testcase-generation interactions were drafts rather than final designs. Pool A exposed the largest oracle problem because the model turned unspecified search behavior into strict pass/fail rules. Pool B showed that a plausible state-machine answer can still contradict a final state or miss matrix edges and concurrency. Pool C showed a similar tendency to invent business validation for decimal prices, URL format, and name uniqueness. Human review corrected 27 candidates and added five cases that the AI missed.

The agent-flow interaction was accepted because it gives human review an explicit place and separates coverage generation from oracle construction and execution planning. The direct evidence path added from the normalized model to the Oracle Builder makes requirement traceability clearer. Overall, AI was most useful for systematic brainstorming and initial structure. It was not reliable enough to own expected results, state rules, or final security conclusions without specification tracing, execution evidence, and student review.

5. Mandatory disclosure

Claude generated the initial 105 API testcase candidates and reviewed the proposed AI-driven generator flow. AI also assisted with organising and wording the audit documents and reusable [SKILL.md](#). The student audited every generated case, assigned the final VALID / INVALID / INCOMPLETE classifications, corrected all 27 invalid or incomplete candidates, added five new cases, executed the Postman collections, and reviewed

the observed failures. Correct expected results were not changed merely to obtain passing executions. CI/CD and Newman screenshots are real execution evidence and were not fabricated. The student remains responsible for the final test design, defect conclusions, manually drawn architecture diagram, and submitted report.

Signature

- **Student name (printed):** Nguyen An Nghiep
- **Student ID:** 23127234
- **Class / Cohort:** Software Testing - 23KTPM1
- **Course:** CS423 / CSC13003 - Software Testing
- **Date:** 19/08/2026
- **Signature:**



The image shows a handwritten signature in black ink on a light gray checkered background. The signature consists of the letters 'A', 'N', and 'N' in a stylized, cursive font, followed by a long horizontal stroke. Below the signature, the name 'Nguyễn An Nghiệp' is written in a similar cursive script.

Reference

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Microsoft. Playwright documentation.
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.

- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.V